Distributed System Documentation: EVCharging

Nicolás Ildefonso Sirvent Orts and Sergio Pascual Morcillo

June 14, 2026

Abstract

This document details the architecture, data modeling, communication protocol, and fault tolerance strategies implemented in the `EV_Charging` project. It is an event-driven distributed system designed for the real-time management, simulation, and monitoring of an electric vehicle charging network. The solution is built around a main orchestrator (`EV_Central`) and peripheral modules that model the supply hardware (`EV_CP_E`), its secure supervision (`EV_CP_M`), the driver's application (`EV_Driver`), device registration (`EV_Registry`), the web interface (`EV_Front`), and the environmental context (`EV_W`).

At the network level, the ecosystem employs a hybrid model that combines Apache Kafka for highly concurrent asynchronous telemetry and TCP/IP *sockets* with an application-level protocol for low-latency synchronous control. Finally, the design places a special emphasis on resilience and cybersecurity, integrating local crash recovery, deferred persistence, Mutual TLS (mTLS) authentication, access delegation via JWT, and end-to-end symmetric encryption, thus guaranteeing the confidentiality and integrity of operations.

Contents

1 Introduction and Objectives

Electric mobility is currently presented as one of the main solutions to reduce polluting emissions and move towards a more sustainable transport model. The constant growth of the electric vehicle fleet requires the deployment of a wide network of charging stations, whose management is critical to ensure the availability, reliability, and efficiency of the service.

In this context, the "EVCharging" project consists of the development of a distributed system designed for the simulation, management, and real-time monitoring of an electric vehicle charging network. The solution addresses the concurrent interaction of multiple system actors: an operations center, the charging points themselves (CPs), and the applications of the drivers requesting the energy supply.

From a technical and academic perspective, the main objective of the project lies in the application and integration of different network communication paradigms. To achieve this, the system's architecture evolves by combining low-level link technologies, through the use of sockets, with high-level platforms oriented towards real-time event *streaming* and the use of message queues. Additionally, the project adopts the principles of a Service-Oriented Architecture (SOA), implementing communication interfaces based on REST API services for the integration of its different components.

Finally, the development aligns with the demands of modern enterprise environments, placing special focus on the resilience, scalability, and security of the distributed system. In this regard, the platform implements control mechanisms for external contingencies, such as evaluating weather conditions through third-party providers, and establishes a robust security framework that guarantees node authentication, communication channel encryption, and constant auditing of critical system events.

2 System Architecture

The EVCharging system has been designed following an event-driven and microservices-oriented architecture. The main modules that make up the ecosystem are detailed below.

2.1 EV_Central

The `EV_Central` module acts as the main orchestrator and decision-making core of the distributed ecosystem. Given the high concurrency inherent to the system (simultaneous reception of telemetry, REST requests, and operational commands), a highly modular internal architecture has been implemented. This separation of responsibilities isolates network logic, volatile state, asynchronous processing, and data persistence in the following subcomponents:

2.1.1 Interface Layer and REST API (`api`)

Developed on the asynchronous *framework* FastAPI, this layer acts as the synchronous gateway of the system. Its goal is to consolidate and serve information to the monitoring dashboards, as well as to receive commands from external systems.

- **Node Management (`cps.py`):** Exposes services for the lifecycle administration of charging points. A highlight is the implementation of the weather *endpoints* (`/weather-alert`), which act as reactive triggers: upon receiving an external alert, they modify the node's memory state and immediately inject an encrypted stop message (`stop`) into the Kafka *broker*, ensuring a real-time response to weather contingencies.
- **Transaction Consolidation (`transactions.py` and `drivers.py`):** These routers solve the problem of eventual consistency. When queried, they combine consolidated historical records in the database (via SQLAlchemy) with volatile, high-frequency telemetry state

stored in memory. This allows the user interface to display the exact consumption and cost up to the current millisecond without saturating the database with continuous queries.

- **Traceability (`events.py`):** Provides read-only access to the historical log of security and operational events.

2.1.2 State and Concurrency Manager (`state`)

The performance of `EV_Central` depends on its ability to process thousands of events per minute without I/O blocking. For this, a highly optimized hybrid state manager (memory-disk) has been designed:

- **Instance Control (Singleton Pattern):** Critical entities like `Database`, `KafkaManager`, and `RegistryKey` implement the *Singleton* pattern supported by logical locks (`Lock`). This ensures that, regardless of the number of active execution threads, the system maintains a single pool of connections to the *broker* and database, avoiding memory leaks and port saturation.
- **Deferred Synchronization (`CPCollection`):** The charging point dictionary operates entirely in RAM (`CPInfo`). To guarantee data durability against unforeseen crashes, `CPCollection` instantiates a daemon thread (`Timer`) that asynchronously and non-blockingly synchronizes the operational state of the chargers with the relational database every 60 seconds.
- **Node Security (`CPInfo`):** Each object in memory encapsulates the symmetric cryptographic key (`Fernet`) dynamically assigned to that specific charging point during its registration phase, keeping the secret isolated in the server's volatile memory.

2.1.3 Event Engine and Business Logic (`kafka`)

This package manages the asynchronous subscription to Kafka *topics*, processing the bulk of the domain's critical operations:

- **Supply Orchestration (`kafka_subscribers.py`):** Centralizes the transaction state machine. For example, the `driver_request_handler` method executes a sequential validation: verifies the CP's existence, checks its availability in memory, issues lock commands (`lock`) to prevent *race conditions*, initializes the formal record in the database, and finally authorizes the physical start of the charge. All this is accompanied by a constant flow of granular notifications to the driver's application.
- **Decryption Layer (`kafka_encripted_subscribers.py`):** To prevent interception and manipulation attacks (*Man-In-The-Middle*), this module acts as a wrapper or *middleware* layer. It intercepts the sealed messages (`EncryptedMessage`) coming from the *Engines*, extracts their payload using the key stored in memory, and transfers the plaintext message to the business handlers, ensuring that the application logic remains agnostic to the encryption layer.

2.1.4 Fault-Tolerant Audit System (`audit`)

The `audit.py` module materializes the system's active security requirements. It transactionally records any relevant event (connections, alerts, rejections). Its architectural design incorporates a strict *fail-safe* control block: if the database engine becomes inaccessible, the error is caught and purged locally, preventing a failure in log persistence from causing an abrupt interruption of the central's critical communication or orchestration services.

2.2 EV_CP_E (Engine)

The EV_CP_E module emulates the low-level hardware and controller of a physical charging point. Its main responsibility is to execute the energy supply, interact physically with the driver, and maintain secure communication with both its local monitor and the central. Given its highly concurrent nature, its architecture is divided into the following functional blocks:

2.2.1 Entry Point and Concurrency (main.py)

The main file acts as the orchestrator of the process lifecycle, managing multiple execution threads to avoid system blocking:

- **Thread Segregation (*Threading*):** The system spins up the socket server (SocketServer) to listen to the monitor and launches the main event loop (run_engine_loop) in a dedicated secondary thread. This allows the Main Thread to be reserved exclusively for the asynchronous rendering of the User Interface (EngineApp).
- **Graceful Shutdown:** Implements signal trapping at the OS level (SIGINT, SIGTERM). Upon receiving an interrupt signal, the system does not terminate abruptly; it injects a SHUTDOWN event into the queue. This allows the engine to safely stop the Kafka consumers, close the network ports, and flush the final state to disk using the CheckpointManager before freeing memory.

2.2.2 Finite State Machine (states Folder)

To manage transactional complexity without incurring fragile conditional logic, the engine implements the State Pattern. The charger's lifecycle flows through the following phases:

- WaitingForConfigState **(Security Isolation):** The restrictive initial state. The charger disables any communication with the central (Kafka) until the Monitor (EV_CP_M) connects via socket and transfers the configuration credentials (Identifier, Cryptographic Key, and Rate).
- IdleState **(Rest and Recovery):** The charger is available. In its initialization phase, it evaluates if there is a previous memory dump (*checkpoint*); if it detects a prior crash with a pending supply, it liquidates the transaction by sending the final *ticket* to the central. During normal operation, it implements a strict timer (AUTHORIZATION_TIMEOUT): if the central authorizes a charge but the user does not plug in the vehicle within 5 seconds, the transaction is automatically invalidated.
- SupplyingState **(Active Supply):** Critical state where the simulated electrical flow occurs. It deploys the telemetry thread and manages asynchronous commands. If it receives a stop order (STOP_ORDER) due to weather conditions while charging, the system does not cut the power abruptly; it activates a flag (pending_stop) to orderly transition out of service once the user finishes their session.
- StoppedState **and** BrokenState **(Contingency):** States of inoperability. The first is dictated by the central (e.g., extreme weather) allowing remote resumption. The second simulates physical hardware failures, forcing the immediate issuance of billing *tickets* and requiring human intervention (simulated via the interface) for its repair.

2.2.3 Telemetry and Persistence (telemetry_thread.py and CheckpointManager.py)

- **Telemetry Daemon Thread:** During the SupplyingState, a secondary thread emulates the electric meter. It incrementally updates the SupplyData model by calculating the cost

(kW x rate) and transmits encrypted messages (`EncryptedMessage`) to the central every second.

- **Crash Recovery Mechanism:** The `CheckpointManager` operates asynchronously, saving JSON memory dumps to the physical disk. This provides the node with *fault tolerance*, ensuring that a driver's billing data is not lost in the event of container crashes or blackouts.

2.2.4 Communications Layer and UI

- **User Interface (`ui.py`):** Using the `Textual` terminal library, it exposes an interactive control panel that injects events (plug in, break down, finish) directly into the `CPEngine`'s queue, emulating hardware interaction and redirecting standard output (`stdout`) to maintain clean observability.
- **Local Communications (`monitor_handler.py`):** Defines the *callbacks* for the `SocketServer`. It acts as a strictly local Command and Control (C&C) bridge, responding to health requests (`STATUS`) and accepting disablement orders (`OUT_OF_SERVICE`) dictated by its supervisor process.

2.3 EV_CP_M (Monitor)

The `EV_CP_M` module acts as the supervisor process (*watchdog*) and perimeter security agent of the charging point. Its primary function is to ensure that the hardware (*Engine*) operates under optimal and secure conditions, isolating the cryptographic provisioning logic and health polling from the charge control flow. The architecture of this component is divided into the following key blocks:

2.3.1 Security and Provisioning Layer (`api_consumer.py` and `central_client.py`)

The Monitor is the only component of the charging point authorized to negotiate the node's identity with the external network. It implements a two-phase security lifecycle:

- **Registration via Mutual TLS (mTLS):** To join the network, the Monitor consumes the REST API exposed by `EV_Registry`. This communication is carried out under the HTTPS protocol requiring mutual authentication; the client validates the central's Certificate Authority (CA) and, in turn, presents its own cryptographic certificate (`cp.crt` and `cp.key`). If registration is successful, the module receives a JSON Web Token (JWT) that it persistently stores on the physical volume (`/data/jwt.token`).
- **Session Authentication (`central_client.py`):** Once registered, the Monitor establishes a socket connection with `EV_Central`, sending its identifier (`CP_ID`) and its JWT signature. Upon validating the token, the central returns the symmetric encryption key that the node will use to encrypt telemetry.

2.3.2 Supervision and Fault Tolerance Engine (`monitor_engine.py`)

The operational core of the Monitor lies in its daemon Polling Thread, which runs in a continuous loop (with a configured interval of 1.0 second). This thread implements mitigation strategies against network contingencies:

- **Downward Polling (Towards the Engine):** Periodically queries the state of the `EV_CP_E` via its local client (`engine_client.py`). If the local socket connection is lost, the Monitor deduces a hardware crash and proactively notifies the central of the breakdown state (`Broken Down`), in addition to persistently attempting reconnection.

- **Upward Polling (Towards the Central):** Forwards the hardware's health state to `EV_Central`. If the central does not respond (orchestrator crash), the Monitor enters self-protection mode: it injects an `OUT_OF_SERVICE` command to the local *Engine*. This ensures that any active supply finishes in a controlled manner and prevents the start of new transactions without network supervision.

2.3.3 Local C&C Controller (`engine_client.py`)

Acts as a strictly internal Command and Control (C&C) bridge. Once the Monitor has successfully negotiated authentication with the central, it uses this socket client to inject the vital configuration into the *Engine*'s memory (`CONFIG#<cp_id>#<cp_key>#<price>`). This deferred injection ensures that the hardware never contains cryptographic material until authorized by the network.

2.3.4 Administration User Interface (`ui.py` and `main.py`)

The Monitor's entry point brings up an asynchronous Textual User Interface (TUI) developed with `Textual`. This interface provides a robust administrative dashboard that allows:

- Manually orchestrating the node's lifecycle (Connect, Register, Authenticate, Unregister) via UI events that delegate blocking tasks to asynchronous *workers*.
- Viewing system logs (*Event Log*) in a *thread-safe* manner via an internal messaging system, isolating UI rendering from network I/O operations.

2.4 EV_Driver

The `EV_Driver` module represents the client application aimed at the end-user (driver). Its design poses a significant architectural challenge: it must maintain an interactive and blocking terminal interface (keyboard reading) while asynchronously processing streams of events coming from the central via Kafka. To resolve this duality, the system abandons traditional sequential flow in favor of an Event-Driven Architecture.

The source code is organized into the following fundamental components:

2.4.1 Phase Orchestration Engine (`main.py` and `Driver.py`)

The application's lifecycle is governed by an implicit state machine that orchestrates the transaction phases. The main controller sequentially evaluates the following stages:

- **Phase 0 (Recovery):** Initialization routine that checks for the existence of a previous suspended state. If it detects an active supply aborted in a previous execution, it executes an architectural *bypass*, skipping the selection and connection phases to directly instantiate the telemetry consumers and resume the charge.
- **Phase 1 (Interactive Selection):** Enables persistent consumers (like the active CP catalog) and displays the selection menu, waiting for user decision or real-time network updates.
- **Phase 2 (Connection Negotiation):** Attempts to establish the session with the selected CP by sending a `SupplyRequestMessage`. It implements a single-attempt policy (eliminating the concept of automated *Round-Robin*) to return control and decision-making capacity to the driver if the connection is denied.

- **Phase 3 (Telemetry and Billing):** Active supply period. The system dynamically recreates the Kafka consumers associated with the session to avoid race conditions or deadlocks derived from the `stop_polling()` method, ensuring clean listening threads in each new transaction.

2.4.2 Event Bus Pattern (`system/event_bus.py` and `events.py`)

To completely decouple the network layer (Kafka) from the user interface and business logic, a centralized Event Bus based on a concurrent queue (`queue.Queue`) has been implemented:

- **Message Standardization:** Any system input (whether a keystroke or a network message) is encapsulated in a `SystemEvent` object typed via the `Intention` enumeration (e.g., `KEYBOARD_INPUT`, `TELEMETRY_INFO`).
- **Non-blocking Filtering:** The `wait_for_events` function acts as a multiplexer. It consumes events from the queue and evaluates whether they match the intentions expected in the current phase. Irrelevant events at that exact moment are re-injected into the queue, implementing a small suspension (0.25s *sleep*) to prevent *spin-lock* scenarios that would lead to *CPU starvation*.

2.4.3 Interface Concurrency (`system/TerminalHandler.py`)

Standard input reading in Python (`input()`) blocks the execution thread. To prevent the driver from missing telemetry or central alerts while deciding their next action, the `TerminalHandler` confines keyboard reading to a secondary daemon thread. When the driver presses a key, this thread captures the value, packages it as a `SystemEvent`, and injects it into the Event Bus in a *thread-safe* manner.

2.4.4 Local Fault Tolerance (`system/FileHandler.py`)

The driver client incorporates a state retention mechanism to survive unexpected shutdowns (such as a forced Docker container shutdown):

- **Signal Interception:** The `Driver.py` module registers callbacks for OS-level signals (`SIGINT`, `SIGTERM`).
- **State Freezing:** Upon capturing a termination signal, the system evaluates if a supply is in progress (`supply_id`). If so, it invokes the `FileHandler` to flush this identifier to physical storage before destroying the process, allowing *Phase 0* to resume operations on the next boot.

2.5 EV_Registry

The `EV_Registry` module plays the role of Identity Provider and the architecture's security gateway. Its exclusive purpose is to govern hardware infrastructure access to the distributed network, ensuring that only legitimate and physically authorized charging points can interact with the central orchestrator (`EV_Central`).

As an independent microservice, its source code is structured into the following logical blocks oriented towards security and persistence:

2.5.1 Authentication and Security Layer (mTLS and JWT)

Network trust management is centralized in the `auth.py` module, which implements a dual-layer perimeter security strategy:

- **X.509 Certificate Validation:** To prevent *spoofing*, the system requires Mutual TLS (mTLS) Authentication. When a monitor requests access, `EV_Registry` decodes the presented certificate in PEM format, cryptographically validates its signature using the public key of the internal Certificate Authority (`ca.crt`), and verifies that the *Common Name* (CN) of the certificate matches the identifier (`cp_id`) requesting registration.
- **Token Issuance (JWT):** Once the physical identity of the equipment is validated, the registry issues a JSON Web Token (JWT) signed with a static key injected securely via OS secrets (`/run/secrets/jwt_secret.key`). This token will act as a passport for the node's subsequent communications with the central.

2.5.2 REST Services Interface (`routes.py` and `main.py`)

The service is exposed through a REST API developed with FastAPI, strictly configured to operate over an encrypted channel (HTTPS) using server certificates (`server.crt` and `server.key`):

- **Registration Endpoint (`POST /registry/cp`):** Exposes the onboarding mechanism. Receives the payload with the identifier, location, rate, and base64 certificate. If the cryptographic validation is successful and the equipment does not incur a uniqueness conflict (HTTP 409), it initializes the charger's state as `DISCONNECTED` in the database and returns the JWT.
- **Deregistration Endpoint (`DELETE /registry/cp/{cp_id}`):** Allows formal unlinking of the equipment, purging its records from the relational database to revoke its access to the network.

2.5.3 Persistence Layer (`database.py` and `models.py`)

To ensure the consistency of data shared with `EV_Central`, the module has its own ORM connection (SQLAlchemy) to the relational engine:

- **Session Management:** Uses the dependency injection pattern (`Depends(get_db)`) to provide transactional and efficient database sessions to each web request, ensuring connections are closed after each operation.
- **Entity-Relationship Model:** Declaratively defines the structure of the `CP` table, mapping native SQL types (`String`, `Numeric`, `Enum`) to Python properties. Furthermore, the main module (`main.py`) implements an initialization routine that ensures automatic table creation (`metadata.create_all`) during the container's boot phase.

2.6 EV_Front

The `EV_Front` module constitutes the presentation and visual monitoring layer of the architecture. Its goal is to provide network operators with an interactive Dashboard to observe the system's state in real-time and issue remote administrative commands.

The module's design does away with complex reactive frameworks, opting for a lightweight microservices-oriented architecture based on **Node.js** for the web server and standard web technologies (HTML5, JavaScript, and CSS via Bootstrap) for the client. Its structure is divided into the following components:

2.6.1 Application Server (`server.js`)

Developed on the Node.js runtime environment using the Express library, this component acts as a static file server that exposes the public directory. Its main technical innovation lies in resolving the environment configuration issue:

- **Dynamic Environment Injection:** Since JavaScript code executed in the browser has no access to the OS or Docker container's environment variables, the server intercepts the virtual route `/config.js` and generates a file on the fly. This script injects the `CENTRAL_API_URL` variable into the global `window` object, allowing the frontend to point to the correct IP and port of the central orchestrator without needing to rebuild the web image for each deployment.

2.6.2 HTTP Client Logic (`app.js`)

The dynamic behavior of the application is managed via a periodic Polling loop executed asynchronously every 2000 milliseconds. Its responsibilities include:

- **Asynchronous REST API Consumption:** Uses the `Fetch` API (`async/await`) to simultaneously query the different central *endpoints*: charger transactional state, supply history, drivers, and audit log.
- **Operational Command Management:** Implements asynchronous calls to execute critical actions. It enables operators to interrupt or resume services, either selectively towards a single node or globally (`stop-all`). Additionally, it includes the ability to simulate attack vectors, allowing manual revocation of an equipment's cryptographic key (`deleteKeyCP`) to force its re-authentication.

2.6.3 User Interface (`index.html`)

The visual layer is implemented applying Responsive Design principles using the Bootstrap 5 library. It consolidates telemetry into three operational sections:

- **Node Grid:** Displays each charging point in an individual card format. The interface dynamically mutates its semantic attributes (colors and labels) based on the charger's remote state machine (available, supplying, broken down, or stopped). It includes direct visual notifications based on weather thresholds.
- **Transactional Panel:** A dynamic data table that correlates driver identifiers with active nodes, facilitating client-side filtering to quickly discriminate between the global history and currently ongoing supplies.
- **Audit Console:** A viewer that emulates standard terminal output, rendering the historical event log. It integrates a parametric URL filtering system that allows specific searches by source address (IP) or action type.

2.7 Weather Control (EV_W)

The `EV_W` module acts as an external agent or sensor that introduces environmental context into the distributed network's decision-making. Its objective is to evaluate the operational viability of charging points based on the real temperature of their geographic location, forcing safety stops in freezing scenarios (temperatures below 0°C).

To meet dynamic evaluation requirements without stopping the system, its architecture is based on a shared memory concurrency model, separating the user interface from the network engine:

2.7.1 Concurrency and Shared Memory Manager

Given that the module must continuously poll APIs while simultaneously allowing keyboard data entry, thread segregation is implemented:

- **Main Thread (Control Interface):** Executes a blocking Command Line Interface (CLI) (`interactive_menu`). This interface allows the operator or evaluator to dynamically map node identifiers (`CP_ID`) to real city names at runtime, without needing to restart the service or the central orchestrator.
- **Daemon Thread (Polling Engine):** A background sub-process (`weather_daemon`) that operates autonomously. Being configured as a *daemon*, the OS guarantees its automatic destruction if the main thread (the menu) terminates.
- **Mutual Exclusion (*Mutex*):** To avoid race conditions when the professor adds a new city and the daemon attempts to read the list simultaneously, the location dictionary (`cp_locations`) is protected by a logical lock (`threading.Lock()`), ensuring that reads and writes are atomic and safe operations.

2.7.2 API Integration and Transactional Logic

The polling engine operates via an infinite loop subjected to a strict 4-second delay (*sleep*) per cycle, thus respecting the external weather provider's Rate Limiting. Its integration flow covers two fronts:

- **Weather Provider (OpenWeather):** Consumes the external API via HTTP `GET` requests, extracting the thermal value from the returned JSON payload.
- **Notification to the Orchestrator (EV_Central):** To avoid saturating the internal network by sending redundant alerts every 4 seconds, the module implements a simplified state machine (`is_frozen`). The system only issues HTTP requests towards the central's `/weather-alert` endpoint when it detects a state transition (from Normal to Frozen via a `POST` method, or from Frozen to Normal via a `DELETE` method). Regardless of alerts, the module uses the `PATCH` method asynchronously to update the temperature in real-time on the central's Dashboard.

3 Data Modeling (Database)

The system's persistent storage is centralized in a relational database (MariaDB). To abstract interaction with the database engine and guarantee code portability, the persistence layer was implemented using the Object-Relational Mapping (ORM) **SQLAlchemy 2.0**.

Data schema management is declarative. Through the foundational class `Base` (which inherits from `DeclarativeBase`), the system can auto-generate the Data Definition Language (DDL) and dynamically build all tables during service boot via routines defined in the `init.py` file (`Base.metadata.create_all`).

3.1 Main Entities and Relationships

The data ecosystem is based on four main entities, strictly typed via the `Mapped` and `mapped_column` structure:

- **Table `CP` (Charging Point):** Represents the inventory of physical charging points.
  - **Key attributes:** Its primary key is an alphanumeric identifier (`String(6)`). It stores contextual information like location (`String(30)`), applied rate (`Numeric(3, 2)`), and local temperature in degrees Celsius (`Float`).
  - **Relational state machine:** The charger's operational state is restricted at the database level using an enumeration (`SQLEnum`) linked to the `CPStatus` class (*Active, Stopped, Supplying, Broken Down, Disconnected*).

- **Table `DRIVER`:** Lightweight entity that consolidates the register of authorized drivers in the system. Its only field is the primary key `id` (`String(9)`), and it maintains a bidirectional One-to-Many relationship with its transaction history.
- **Table `SUPPLY` (Transactions):** This is the core transactional entity of the model. It acts as a semantic link table between a `Driver` and a `CP`, recording the lifecycle of an electrical supply.
  - **Volatile lifecycle:** At the start of the supply, the `price` and `consumption` fields allow null values (`nullable=True`), delegating their storage to the close of the session. Progress is tracked via the boolean flag `is_done`.
  - **Integrity Constraints:** Implements two Foreign Keys. Of special architectural interest is the constraint applied to `cp_id`: it is configured with the `ondelete="SET NULL"` policy. This ensures that if the `EV_Registry` deregisters and physically deletes a charging point from the database, the billing history (`Supply`) is not destroyed in a cascade, preserving the integrity of completed charges.
- **Table `EVENT` (Audit):** Supports the security and traceability component of the system.
  - **Attributes:** Records a timestamp delegated to the server engine (`func.now()`), the source IP address of the request (`String(15)`), the action type (`String(50)`), and a detailed description in plaintext (`Text`). By lacking restrictive foreign keys, it ensures very low-latency insertions, ideal for auditing logs in high-concurrency systems.

4 Communication Protocol

The distributed ecosystem requires constant, asynchronous, and fault-tolerant information exchange. To achieve this, the system relies on Apache Kafka for event-based messaging, and direct TCP/IP connections (*sockets*) for low-latency administrative commands.

4.1 Internal `communications` Library

To abstract the complexity inherent in low-level network and messaging libraries (such as `confluent_kafka` and the native `socket` library), an internal package or wrapper called `communications` has been developed. This approach guarantees code cohesion, facilitates dependency injection, and standardizes communications across all project modules.

4.1.1 Event Layer (Kafka)

The Kafka submodule has been designed strictly applying SOLID principles and software design patterns to manage high performance and concurrency:

- **Serialization Contract (`Message.py`):** Defines an abstract base class (ABC) that forces all network entities to implement serialization (`to_payload`) and deserialization (`from_payload`) methods. This ensures the Kafka engine remains agnostic regarding the semantic or business structure of the data it transports.
- **Factory Pattern (`KafkaFactory.py`):** Centralizes the instantiation of producers and consumers. By encapsulating the *broker* configuration (`KafkaBrokerInfo`), business modules can request communication channels without knowing the details of the underlying infrastructure.
- **Synchronous Producers (`KafkaProducer.py`):** Implements message sending ensuring delivery via immediate network buffer flush (`flush()`), guaranteeing minimal dispatch latency.

- **Asynchronous Consumers (`KafkaConsumer.py`):** Message consumption is inherently blocking. To avoid halting application processes, the consumer instantiates its own background daemon thread. This thread performs periodic polling (`poll(1.0)`), extracts the content (*payload*), dynamically deserializes it according to the injected class, and applies strict filtering functions (`filter_func`) to discard irrelevant traffic (for example, telemetry directed to other identifiers).
- **Observer Pattern (`KafkaNotifier.py`):** To avoid direct coupling between the network layer and business logic, a generic notifier is implemented. Application handlers (*subscribers*) subscribe to this notifier and are invoked reactively (via callbacks) only when the consumer thread processes a valid event.

4.1.2 Synchronous Communications Layer (TCP Sockets)

For low-latency Command and Control (C&C) operations, as well as authentication processes and credential transmission between physical nodes and the orchestrator, the library implements a synchronous, stateful network layer based on TCP/IP *sockets*.

To avoid the inherent fragility of transmitting raw bytes, the design encapsulates the logic in four main components that implement a rigorous application layer protocol:

- **Frame and Protocol Management (`SocketConnection.py`):** This class implements a deterministic protocol for message exchange, guaranteeing data integrity against network latency or fragmentation:
  - **Initial Handshake:** Session establishment through the exchange of control bytes `<ENQ>` (Enquiry) and `<ACK>` (Acknowledge) or `<NACK>` (Negative Acknowledge).
  - **Framing:** Messages are delimited by injecting a Start of Text byte (`<STX>`) and an End of Text byte (`<ETX>`).
  - **Error Control (LRC):** Implements a Longitudinal Redundancy Check. The sender calculates a parity block by applying a bitwise XOR operation over the payload and appends it to the frame. The receiver recalculates the LRC locally and issues an `<ACK>` only if both signatures match.
  - **Controlled Closure:** Session termination via the transmission of the `<EOT>` (End Of Transmission) byte.
- **Client-Server Architecture (`SocketServer.py` and `SocketClient.py`):** The server adopts a concurrency model based on one thread per connection (*Thread-per-Connection*) in daemon mode, allowing it to attend multiple monitors simultaneously without blocking the main execution thread (`accept_loop`). The client, on the other hand, incorporates resilience logic, applying transparent automatic reconnections if a frame transmission fails. Both components are designed to inject an `ssl.SSLContext`, elevating the raw socket to an encrypted tunnel (TLS) when system security requires it.
- **Dynamic Routing (`MessageHandler.py`):** To avoid the anti-pattern of nested conditional statements (*if-else chaining*) when analyzing received commands, a Router or Command pattern is implemented. This manager extracts the message header (delimited by default by the `#` character) and delegates business logic execution to dynamically registered callback functions in a dictionary.

4.2 Communication in EV_Central

The `EV_Central` module acts as the concentrator node for messaging traffic across the entire system. Its communication protocol over Kafka is structured by the strict definition of message objects, which type and standardize bidirectional interactions with drivers and charging points.

4.2.1 Messaging Catalog (Kafka)

Asynchronous communication materializes through specialized classes that inherit from the abstract class `Message`. This inheritance ensures compliance with the serialization contract (`to_payload`) to JSON format and its respective deserialization (`from_payload`). The catalog is divided by logical domain:

- **Supply Orchestration:**
  - `SupplyRequestMessage`: Receives charge requests issued by drivers, encapsulating the destination node and source IP for centralized audit logging.
  - `SupplyResponseMessage`: Notifies the driver's application of the resolution of their request (accepted or denied) along with the reason and the unique identifier of the transaction in the database.
  - `StartSupplyMessage`: Critical low-level instruction sent to the physical *Engine* to authorize the release of the electrical supply associated with a transaction.
- **Telemetry and Global Synchronization:**
  - `SupplyTelemetryMessage`: Acts polymorphically according to its `msg_type` attribute. If its value is `"supplying"`, it transfers incremental updates of consumption (kWh) and cost. If it's `"ticket"`, it issues the final remittance that closes the financial transaction.
  - `ActiveCPListingMessage`: Proactively emits (*broadcasting*) an updated matrix with the identifiers of all available chargers (`ACTIVE`) towards the drivers' bus.
- **Commands and Notifications:**
  - `CentralCommandMessage`: Conveys imperative instructions from the orchestrator to the *Engine*'s state machine (`stop`, `resume`, `lock`), fundamental for forcing operational stops dictated by the Weather Office.
  - `SupplyRequestNotificationMessage` and `SupplyErrorMessage`: Provide asynchronous, granular feedback to the driver regarding internal state transitions or exceptions in the *broker*.

4.2.2 Cryptographic Middleware: Transparent Encapsulation

To mitigate the risk of interception or injection vulnerabilities in the *broker* network (Man-In-The-Middle attacks), the system implements the `EncryptedMessage` class. This entity operates as a transparent cryptographic wrapper or Middleware:

1. **Origin Encryption:** During serialization, the class queries the in-memory dictionary (`CPCollection`) to extract the symmetric key (`Fernet`) assigned specifically to the destination charging point. It then serializes the internal business payload and encrypts it, injecting it into a new JSON package alongside the `cp_id`.
2. **Dynamic Destination Resolution:** In the deserialization process, `EncryptedMessage` decrypts the payload and reads the underlying `type` attribute. It immediately uses a static resolution map (`_CLASS_TRANSLATOR`) to dynamically instantiate the corresponding original business class (e.g., `CentralCommandMessage`). This allows `EV_Central`'s handlers to operate exclusively on plaintext objects, keeping the core logic completely decoupled from peripheral cryptographic complexity.

4.2.3 Synchronous Communications and Event Propagation (Sockets)

For direct, low-latency administrative interaction with physical monitors (`EV_CP_M`), the central exposes a TCP/IP server using the internal `communications` library. This layer is not limited to answering network requests, but acts as an architectural bridge or transactional Middleware between the synchronous operations of chargers and the asynchronous event bus (Kafka).

The control logic is segmented into two fundamental components:

- **Session and Routing Management (`make_handler.py`):** A closure-based design pattern is used to instantiate a router (`MessageHandler`) that preserves the memory reference of the charging point (`CPInfo`) throughout the entire socket connection lifecycle. This manager implements critical operational flows:
  - **Authentication Phase (`AUTH`):** Intercepts the initial connection frame, extracts the monitor's JSON Web Token (JWT), and cryptographically validates it using the system secret (`RegistryKey`). If the signature and subject (`sub`) are verified, the orchestrator registers the node in the concurrency manager (`CPCollection`), generates a unique symmetric encryption key for the session, and returns it encapsulated in the validation response.
  - **Cross-Event Reactivity (`STATUS`):** Applies a reactive observer behavior. Upon receiving updates of the charger's operational state, it evaluates the implications on the global ecosystem. For example, if a charger transitions to a breakdown state (`BROKEN_DOWN`) while executing a supply, the local handler proactively injects an alert message (`SupplyErrorMessage`) into Kafka's `supply.errors` topic, ensuring the driver receives an instant notification on their terminal app. Similarly, any alteration in an equipment's availability (`ACTIVE`) triggers a broadcast (`ActiveCPListingMessage`) updating station catalogs on `EV_Driver` clients.
- **Loop Controller and Auditing (`handle_monitor.py`):** Every new connection accepted by the TCP server instantiates an independent thread delegating control to this module. Its responsibility is to govern the asynchronous read and response loop (`receive` / `send`) until an orderly closure (`BYE`). Highlights include its tight integration with the security engine (`audit.py`): it transactionally audits each received instruction, recording the action and origin IP address. To prevent I/O saturation in the relational database given the constant sending of monitor heartbeats, the controller implements a filtering heuristic that discards persistence of redundant states, logging `STATUS` frames only when an effective alteration occurs (`last_status` $\neq$ `description`).

4.3 Communication in EV_CP_E (Engine)

The `EV_CP_E` module emulates charging hardware, so its network layer must support the constant emission of telemetry and reception of real-time commands, ensuring the consistency of the simulated physical state.

4.3.1 Transactional Messaging and Telemetry (Kafka)

The *Engine* communicates asynchronously with the central orchestrator using a specific subset of the message catalog, applying an end-to-end encryption paradigm:

- **Integrated Symmetric Encryption:** All inbound and outbound traffic of the charger flows through the `EncryptedMessage` class. Unlike the central (which dynamically manages multiple keys), the engine stores its cryptographic key statically in the class itself

(_cipher_key). This key is injected at runtime by the local Monitor. Any message processed by the engine's producers or consumers is automatically encrypted or decrypted using the *Fernet* algorithm.

- **Egress Flow:** The engine instantiates producers to dispatch authorization requests (SupplyRequestMessa when it detects a driver's interaction. During the charge cycle, it continuously emits SupplyTelemetryMessage objects to report consumption (supplying) and, upon finishing, issues the same typed message format as a ticket for billing.
- **Ingress Flow:** Engine consumers listen passively on their respective *topics*. They await authorization commands (StartSupplyMessage) or control directives (CentralCommandMessage), such as stop or resume orders originating from the Weather Office.

4.3.2 Internal Event Management and Decoupling

To avoid race conditions derived from receiving asynchronous messages on multiple network threads (e.g., receiving a stop command at the exact time a user disconnects the vehicle), the *Engine* implements a strict isolation pattern via an **Internal Event Queue** (queue.Queue):

1. **Reception and Translation:** When a Kafka consumer receives and decrypts a valid message (e.g., a CentralCommandMessage with a stop action), the network handler (kafka_handlers.py) **does not** execute business logic or alter the engine's state directly. Its sole responsibility is to translate the external message into an internal semantic event, instantiating an Event(EventType.STOP_ORDER) object.
2. **Secure Injection:** This internal event is injected into the *Singleton* CPEngine's centralized queue via a *thread-safe* method (put_event()).
3. **Sequential Processing:** The engine's main thread sequentially and blockingly extracts events from the queue (handle_next_event()). It is at this point, free from concurrency, that the Finite State Machine (FSM) evaluates the event and invokes the corresponding transition (e.g., transition to StoppedState).

This architectural design ensures that the *Engine*'s state machine operates purely deterministically, processing network directives and physical interactions (UI buttons) in the strict temporal order they occurred.

4.4 Communication in EV_CP_M (Monitor)

The EV_CP_M module operates as the Security Gateway and health supervisor of the charging point. Its communications protocol is hybrid, combining synchronous HTTP requests for cryptographic provisioning and persistent TCP connections (*sockets*) for real-time operations.

4.4.1 Secure Provisioning: REST API and mTLS (api_consumer.py)

The charger's initial registration phase executes over an HTTPS channel strictly secured by **Mutual TLS (mTLS) Authentication**. This mechanism ensures that neither party blindly trusts the other:

- **Bidirectional Validation:** When consuming the POST /registry/cp endpoint, the requests library is configured to validate the EV_Registry server certificate, checking it against the internal Certificate Authority (CA_CERT). Simultaneously, the client injects its own pair of physical cryptographic keys (cp.crt and cp.key) into the transport layer (cert=CLIENT_CERT) to prove its hardware legitimacy to the central.

- **Credential Exchange:** The request payload transfers operational metadata (identifier, location, rate) and a base64 copy of the public certificate. If `EV_Registry` authorizes the transaction, the monitor receives a JSON Web Token (JWT) that acts as proof of identity (*Bearer Token*) for all remaining interactions.
- **Unregister:** Through a `DELETE` request, the monitor can cleanly revoke its own network access, deleting its records from the central database.

4.4.2 Authentication and Heartbeats with the Orchestrator (`central_client.py`)

Once provisioned, the monitor establishes a persistent TCP/IP tunnel with `EV_Central` using the internal `communications` library:

- **Symmetric Key Exchange (`AUTH`):** The first frame sent to the central encapsulates the node's identifier and the previously obtained JWT (`AUTH#<cp_id>#<jwt_token>`). The central verifies the token's signature and returns the symmetric encryption key (*Fernet key*), which is vital for encrypting telemetry on Kafka.
- **State Reporting (`STATUS`):** Acts as a Heartbeat emitter. Periodically sends the node's operational state (e.g., `Active`, `Broken Down`) to the orchestrator, guaranteeing that the global Dashboard reflects physical incidents within milliseconds.

4.4.3 Local Command and Control (`engine_client.py`)

To isolate the *Engine* (the physical charge simulator) from the public network, the monitor brings up a second socket connection directed exclusively at the local process (*localhost*):

- **Configuration Injection (`CONFIG`):** The hardware boots in an inoperative state (Security Isolation). It is the monitor that, after authenticating with the central, injects its identity, rate, and cryptographic key using the `CONFIG#<cp_id>#<cp_key>#<price>` frame.
- **Hardware Polling:** The monitor continuously queries the engine using `STATUS#` frames to detect process crashes or lockups. If the *Engine* stops responding, the monitor automatically transitions its network state to broken down.
- **Forced Isolation (`OUT_OF_SERVICE`):** Upon detecting a crash in `EV_Central`, the monitor injects this order into the local engine, forcing a safety shutdown that prevents the initiation of new transactions without remote supervision.

4.5 Communication in EV_Driver

The `EV_Driver` module represents the architecture's end client. Its network design must resolve a classic concurrency problem: maintaining the ability to send interactive commands to the orchestrator while asynchronously and constantly receiving telemetry and state updates without blocking the main execution thread.

4.5.1 Messaging Protocol (Kafka)

External communication with `EV_Central` is conducted exclusively via the Kafka event bus, using the standardized messaging catalog from the `communications` library. Data flow is divided into:

- **Egress Flow:** The application acts as a producer of a single message type. When the user selects a charging point, the client emits a `SupplyRequestMessage`, injecting its identifier, the destination (`cp_id`), and its local IP address (for centralized audit logging).

- **Ingress Flow:** The application deploys asynchronous consumers in the background to process network responses:
  - **Discovery:** Receives `ActiveCPListingMessage` objects that update the available station catalog in real-time.
  - **Negotiation:** Processes `SupplyResponseMessage` objects informing of charge acceptance or denial, and `SupplyRequestNotificationMessage` to receive text traces about authorization progress.
  - **Supply and Contingencies:** During charging, it intercepts `SupplyTelemetryMessage` objects to render consumption (`supplying`) or generate the final invoice (`ticket`). It also listens to `SupplyErrorMessage` objects to alert the driver if the physical charger suffers a breakdown (`BROKEN_DOWN`) mid-session.

4.5.2 Concurrency Manager: Internal Event Bus

To prevent coupling between the network layer (Kafka), the user interface (blocked by the `input()` function), and the business logic, the application implements an Event-Driven Architecture supported by a Centralized Event Bus (`event_bus.py`):

1. **Event Taxonomy (`events.py`):** Defines a strict enumeration (`Intention`) categorizing any system stimulus (e.g., `KEYBOARD_INPUT`, `TELEMETRY_INFO`, `DENIED_RESPONSE`). Every stimulus, whether from the network or keyboard, is encapsulated in an immutable data structure (`SystemEvent`).
2. **Asynchronous Translators:** Kafka consumer handlers (e.g., `telemetry_info_handler`) operate as simple translators. They execute no business logic nor print to the screen; their sole function is to receive the Kafka message, package it into a `SystemEvent` with the corresponding intention, and deposit it into the concurrent queue (`EVENT_QUEUE.put()`).
3. **Multiplexing and Filtering (`wait_for_events`):** The driver's main state machine continuously extracts events from the queue. To manage the arrival of out-of-order or irrelevant events for the current phase (e.g., receiving telemetry while still navigating the menu), the extraction function implements a non-blocking filtering mechanism. If the extracted event doesn't match desired intentions, it is re-injected at the end of the queue (*requeue*) and the thread suspends execution briefly (0.25 seconds). This strategic pause (*backoff*) prevents spin-lock scenarios, averting CPU starvation.

4.6 Communication in EV_Registry

Unlike operational modules using asynchronous messaging or raw TCP sockets, the `EV_Registry` operates exclusively as a passive server (API Gateway) based on the HTTP protocol. Its role is to act as a Registration Authority, receiving provisioning requests from charging points.

To explain its communication model, we must break down its transport layer, validation protocol, and interaction with the data model:

4.6.1 Secure Transport Layer (HTTPS and mTLS)

The service is exposed via the FastAPI framework, deployed over the ASGI Uvicorn server (`main.py`). To ensure confidentiality in credential transit, the server is initialized by explicitly injecting its own cryptographic certificates (`server.crt` and `server.key`). This elevates the communication channel to HTTPS.

The protocol requires **Mutual TLS Authentication (mTLS)**. This implies that the client (the Monitor) must not only trust the server but present its own digital identity. This identity travels base64-encoded within the HTTP request payload.

4.6.2 Interaction Protocol (REST Endpoints)

Communication is orchestrated by the `routes.py` router, exposing two transactional interfaces:

- **Device Onboarding (`POST /registry/cp`):** The critical flow of the module. Upon receiving a request, the system executes sequential validation:
  1. **Cryptographic Verification (`auth.py`):** Deserializes the X.509 certificate provided by the client and mathematically verifies its signature using the Certificate Authority's public key (`ca.crt`) via the PKCS#1 v1.5 padding scheme. Next, validates that the certificate's *Common Name* (CN) exactly matches the `cp_id` claimed in the request, preventing spoofing attacks.
  2. **Duplicate Control:** Queries the database to ensure the identifier is not already provisioned (returning `HTTP 409 Conflict` if affirmative).
  3. **Token Issuance:** If validation is successful, generates a JSON Web Token (JWT) algorithmically signed (`HS256`) with a static system secret (`jwt_secret.key`). This token is returned to the client in JSON format.
- **Device Revocation (`DELETE /registry/cp/{cp_id}`):** Destructive request that processes equipment removal. Locates the entity in the database and executes physical deletion (`db.delete()`). Thanks to the database design (`SET NULL` constraint on foreign keys), this action revokes device access without corrupting the supply history at the central.

4.6.3 Data Integration (SQLAlchemy in FastAPI)

The module communicates with the underlying MariaDB database sharing the same relational model as `EV_Central` (`models.py` file). To ensure network operations do not generate blocks or connection leaks to the database, the router uses FastAPI's Dependency Injection pattern (`Depends(get_db)`).

This pattern generates an isolated database session (`SessionLocal`) for each incoming HTTP request, executes `INSERT` (leaving the node in `DISCONNECTED` state) or `DELETE` statements, commits the transaction, and guarantees unconditional closure (`db.close()`) of the connection upon finishing the network response.

4.7 Communication in EV_Front

The `EV_Front` module constitutes the graphical monitoring interface of the system. Operating in the client's web browser, its communications architecture is based on the asynchronous and stateless consumption of RESTful services exposed by `EV_Central`'s API.

4.7.1 Asynchronous Global State Consumption

Since the ecosystem lacks direct *WebSockets* to the frontend, real-time interface updates are guaranteed via a continuous polling pattern (Long Polling / Short Polling).

- **Polling Loop (`setInterval`):** The `fetchData()` function is automatically invoked every 2000 milliseconds. This routine acts as an orchestrator of requests, making simultaneous `HTTP GET` calls to recover the network's updated state.
- **Transactional Grouping:** In a single clock cycle, the frontend queries multiple endpoints:
  - `/api/cps/`: To obtain the temperature, rate, and operational state (`ACTIVE`, `STOPPED`, `BROKEN_DOWN`) of each node.
  - `/api/transactions/`: Recovers live billing, crossing kilowatt consumption (kWh) with supplies marked as active (`!t.is_done`).

- `/api/drivers/`: Obtains the catalog of authorized users.
- **Parametrized Auditing (`/api/events/`):** Demonstrates dynamic filtering capability. Constructs `URL` and `URLSearchParams` objects on the fly to inject filters (`ipFilter`, `actionFilter`) into the query string, offloading processing and log discrimination to the central orchestrator's database engine.

4.7.2 Command Injection and Contingency Control

The graphical interface is not a mere passive viewer; it operates as a high-level command terminal, issuing `HTTP POST` and `DELETE` requests to alter the distributed network's behavior deterministically:

- **Mass Orchestration:** Features global `stopAllCPs()` and `resumeAllCPs()` methods. Upon executing `POST /cps/stop-all` calls, the frontend triggers a chain reaction in the central orchestrator, which propagates stop orders on Kafka's topic to all active charging points, freezing supply network-wide.
- **Granular Control and Fault Simulation:** The UI implements individual node-level functions (`stopCP(id)` and `resumeCP(id)`). As a penetration and resilience test, it incorporates the ability to send `DELETE /cps/{id}/key` directives. This command instructs the central to unilaterally revoke the cryptographic key of a specific *Engine* mid-execution, emulating credential loss or cryptographic compromise to force the Monitor's (`EV_CP_M`) re-authentication protocols.

4.8 Communication in Weather Control Office (EV_W)

The `EV_W` module acts as a bidirectional integration bridge between external information services and the internal distributed network. At the communications level, its design is based on RESTful API consumption via the synchronous HTTP client `requests`, operating in a daemon thread to avoid blocking the user interface (CLI).

This module's network protocol bifurcates into two distinct integration flows:

4.8.1 External Consumption (OpenWeather API)

To obtain climate variables, the module conducts Active Polling against OpenWeather's public API.

- `GET` **Requests:** Dynamically constructs the URL by injecting the mapped city name and access key (`API_KEY`), explicitly requesting the metric system (`units=metric`) to obtain the temperature directly in degrees Celsius.
- **Quota Control (Rate Limiting):** To avoid network saturation and blocking by the external provider due to excess traffic, the communications thread imposes a strict suspension (`time.sleep(4)`) of 4 seconds between each global evaluation cycle.

4.8.2 Internal State Injection (EV_Central API)

Communication with the central orchestrator is designed to minimize redundant traffic, applying a reactive integration paradigm based on in-memory state control (`cp_locations`):

- **Thermal Telemetry Synchronization (`PATCH`):** Regardless of whether an alert exists, the module emits `HTTP PATCH` requests towards the central's `/api/cps/{cp_id}/temperature` endpoint every reading cycle. This ensures the graphical Dashboard passively reflects thermal oscillations in real-time.

- **Critical Alert Transition (`POST`):** If the local temperature drops below 0°C and the node was not previously cataloged as frozen (`is_frozen == False`), the module notifies the anomaly via an `HTTP POST` request to the `/weather-alert` endpoint. This call instructs the central to asynchronously emit a temporary stop order (`stop`) to the *Engine* via Kafka.
- **Contingency Resolution (`DELETE`):** Once the weather surpasses the freezing threshold, the system evaluates the transactional flag and emits an `HTTP DELETE` request to the same alert endpoint. This action revokes the contingency state and returns control to the central to automatically resume the physical service.

5 Fault Tolerance and Security

In a distributed system handling physical and financial transactions in real-time, availability and resilience are critical architectural requirements. The system does not assume the reliability of the underlying network or hardware, implementing mitigation strategies at each of its nodes.

5.1 Fault Tolerance per Module

5.1.1 EV_Central: Deferred Persistence and Error Isolation

The central orchestrator manages thousands of transactions, so a database crash or saturation must not paralyze the communications bus:

- **Asynchronous Persistence:** Instead of blocking network threads by writing every thermal or operational state change to disk, the `CPCollection` class stores these metrics in volatile RAM. A daemon thread (`Timer`) performs a Flush synchronization to the relational database asynchronously every 60 seconds.
- **Fail-Safe Auditing:** The event logging module (`audit.py`) intercepts and internally suppresses any ORM engine (SQLAlchemy) exception. If the MariaDB container restarts or loses connectivity, the orchestrator temporarily discards the log write, ensuring the Kafka loop and synchronous sockets continue serving critical requests.

5.1.2 EV_CP_E (Engine): Crash Recovery

Charging hardware is subject to local power cuts or abrupt operational container restarts. To prevent economic losses from incomplete billing:

- **Checkpoints:** The `CheckpointManager` entity operates decoupled, saving transactional snapshots in JSON format directly to the filesystem volume (`/data/checkpoint.json`) as the supply progresses.
- **Self-Recovery:** During the boot phase (`IdleState`), the engine reads persistent storage. If it detects a supply marked incomplete (`pending_stop` or no completion signal), it restores the state variable in memory and immediately issues the final *ticket* to the central with the last consolidated value before power failure, subsequently purging the physical file.

5.1.3 EV_CP_M (Monitor): Graceful Degradation

The monitor operates under the premise of Network Fallacies. It implements a double Polling cycle with active retry protocols:

- **Local Disconnection:** If the socket communication with the *Engine* is interrupted (process failure), the monitor proactively alerts the central with a `BROKEN_DOWN` state and initiates an infinite reconnection loop.

- **Remote Disconnection:** If the monitor loses connectivity with the orchestrator (`EV_Central` inaccessible), it assumes a Fail-Secure posture. It immediately injects an `OUT_OF_SERVICE` command to the local physical motor. This graceful degradation allows ending any ongoing electrical supply orderly but prevents initiating new unsupervised or unbillable charges until the global network link is restored.

5.1.4 EV_Driver: State Freezing

To protect the driver's experience against client application crashes (abrupt terminal closure or process stop):

- **Signal Interception:** The client captures `SIGINT` and `SIGTERM` signals at the OS level. Instead of terminating execution instantly, it hands control to a shutdown routine using the `FileHandler`. If a charge session is in progress, the client serializes the transaction identifier (`supply_id`) to disk before freeing memory.
- **Boot Bypass:** On the next execution (Phase 0), the application detects the recovery file, skips the initial selection menu, and dynamically re-instantiates Kafka consumers to resume telemetry reception without logical interruption.

5.1.5 EV_W and REST Components: Network Resilience

All synchronous HTTP clients in the ecosystem (OpenWeather, `EV_Registry` calls, or `EV_Front` requests) are instrumented with temporal timeout clauses (`timeout=5`). This prevents network thread freezing in severe latency cases. Given `requests.RequestException` type exceptions, processes catch the anomaly and retry access on the next timed cycle, avoiding cascading service failures.

5.2 Comprehensive Security Strategy

The distributed ecosystem handles sensitive billing data and physical hardware control. To guarantee confidentiality, integrity, and authenticity, the architecture adopts a "Defense in Depth" approach, implementing controls at the network, application, and cryptographic layers.

5.2.1 Public Key Infrastructure (PKI) and mTLS

To secure REST provisioning communications, an automated local Certificate Authority (CA) has been deployed via shell scripting (`gen_ssl_cert.sh` and `gen_cp_cert.sh`).

- **Server Certificates with SAN:** The web server (`EV_Registry`) generates its key pair dynamically injecting its IP address into the Subject Alternative Name (SAN) extension. This practice allows operating the environment in local lab infrastructures, preventing network libraries from rejecting the connection due to strict HostName mismatch discrepancies.
- **Physical Node Identity (mTLS):** Each charging point generates a client certificate (`cp.crt`) whose *Common Name* (CN) attribute is forced via script to match its identifier exactly (e.g., `CP01`). During the bidirectional TLS handshake, the server validates the certificate is signed by the system's CA and, additionally, that the CN corresponds to the equipment attempting registration, effectively mitigating spoofing attacks.

5.2.2 Session Control and Secret Isolation

The architecture prohibits the transit of plaintext passwords.

- **Token Authorization:** Once physical identity is validated, the registry issues a JSON Web Token (JWT) signed under the `HS256` algorithm. This token is stored on the local equipment volume and acts as proof of delegated authorization (*Bearer*) to establish persistent TCP connections with the central orchestrator.
- **Secret Injection:** Master keys, such as the JWT signature secret, are injected into containers exclusively at runtime via the secure filesystem (`/run/secrets/`), preventing accidental exposure of cryptographic material in source code repositories.

5.2.3 Symmetric In-Transit Encryption (Application Layer)

Since the Kafka broker network acts as an intermediate public bus susceptible to eavesdropping, a point-to-point application encryption mechanism (E2EE) is implemented:

- **Dynamic Key Negotiation:** Following successful TCP authentication, the central generates an algorithmically unique symmetric cryptographic key (*Fernet key*) for that session and transfers it securely to the physical monitor.
- **Cryptographic Wrapper:** Network modules route telemetry and operational commands via the `EncryptedMessage` class. This middleware encrypts the payload at origin and decrypts it at destination using the shared key, ensuring that successful interception of Kafka topics by a Man-In-The-Middle (MITM) attack results in illegible and mathematically unalterable information without corrupting the packet.

6 Deployment

In this section, we present a set of steps to follow in order to deploy the different applications of our project using Docker Compose.

1. Create .env files in the root directory of each submodule (for example, the .env file's path of EV_Central submodule should be EVCharging/EV_Central/.env).
2. Copy the contents of .env.example of each submodule into the new .env file and make the appropriate changes, such as IP addresses, ports, etc. The only IPs that must be changed correctly appear in the .env.example as 127.0.0.1.
3. Go to the deployment/real directory and replace the IP address 127.0.0.1 with the IP address of the host in the environment variable `KAFKA_ADVERTISED_LISTENERS`. This is key for Kafka's correct functioning.
4. Go to the certificates directory and execute the `gen_ssl_cert.sh` script in order to create the SSL certificates for the central and registry modules. This script will ask for the IP address of the host machine.
5. For more comfort, change the name of the corresponding Docker Compose file to be executed in a host machine to `docker-compose.yml`.
6. Run the following commands to deploy the different applications depending on the conditions:
   - If the application has no TUI (e.g., MariaDB database, Kafka, Kafka's initialization script, central, registry and frontend), run:

     ```
     docker compose up <service(s) name(s)> -d
     ```
   - If the application has TUI (e.g., monitor, weather control office and driver), run:

```
docker compose run --rm <service name>
```

Run this command for each submodule in different terminal windows.

- To deploy the engine, run the following command:

  ```
  docker compose run --rm --service-ports <engine service name>
  ```

  Run this command for each engine in different terminal windows

The previous steps must be followed in order to deploy the project in a distributed environment like the laboratory. Yet, to test the project in a single machine (like we did in our homes), we have also created a Docker Compose file that emulates this environment using Docker networks. The steps to deploy in a single machine are fundamentally the same as the ones we have exposed before.

6.1 Deploying new charging points and drivers

As the process of creating a new charging point is quite tedious (since a certificate, configuration files and the containers must be created), we have created a shell script (`new_cp.sh` in `EVCharging/deployment`) that automates this process. This script will ask the user about information regarding charging point (id, price, location and host IP address) and the rest of submodules (central, registry and Kafka broker) and will generate Docker Compose file inside a new folder.

Similarly, as the process of creating a new driver manually can also be tedious (since environment variables and an isolated storage volume must be set up), we have created a shell script (`new_driver.sh` in `EVCharging/deployment`) that automates this process. This script will ask the user about information regarding the driver (such as its ID) and the Kafka broker (IP address and port), and will generate a Docker Compose file inside a new folder.

7 Execution screenshots

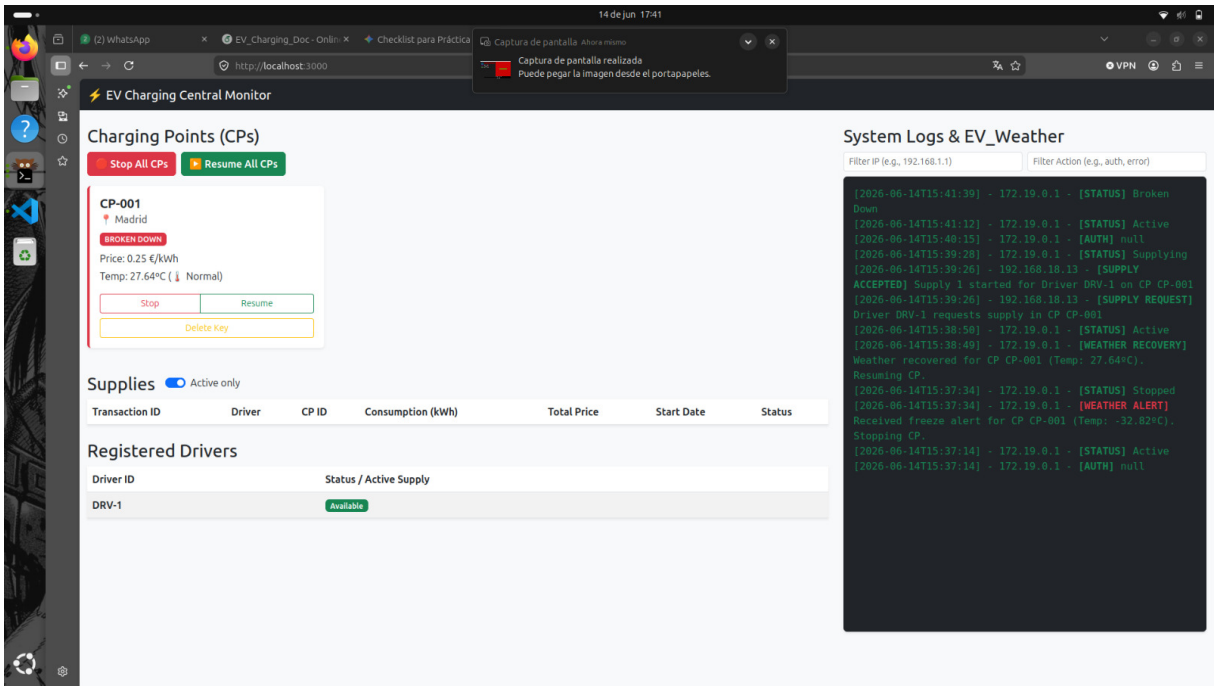

Figure 1: Front Web, supply error

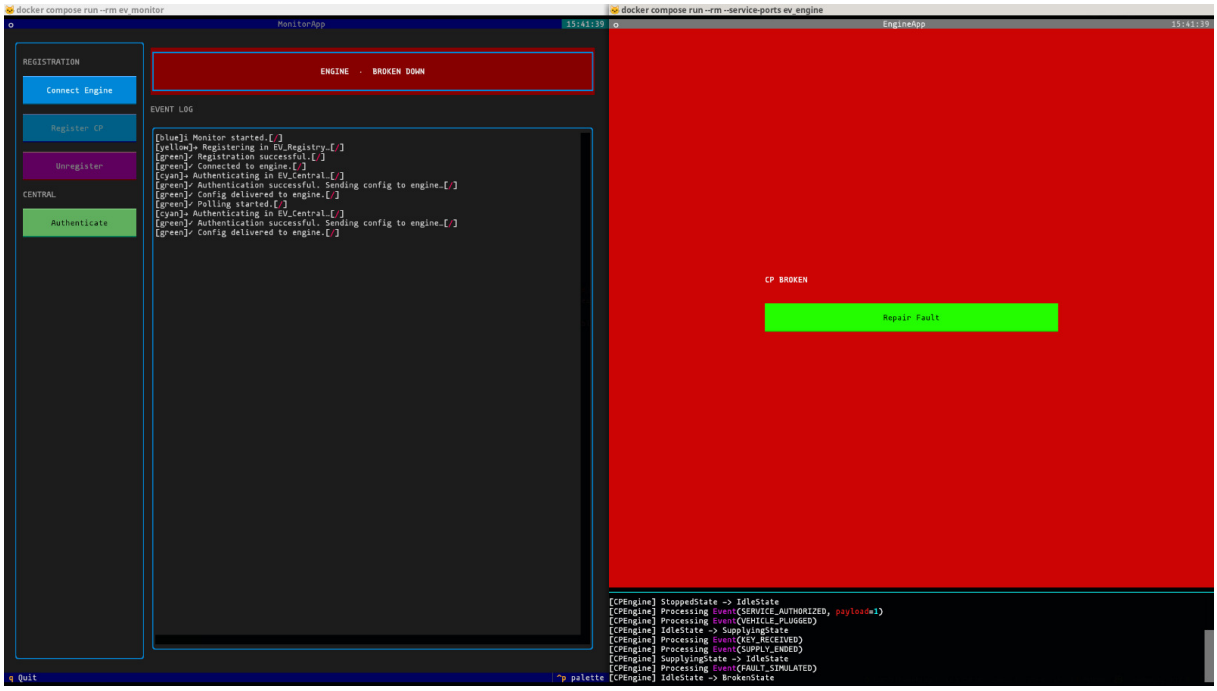

Figure 2: CP simulate error

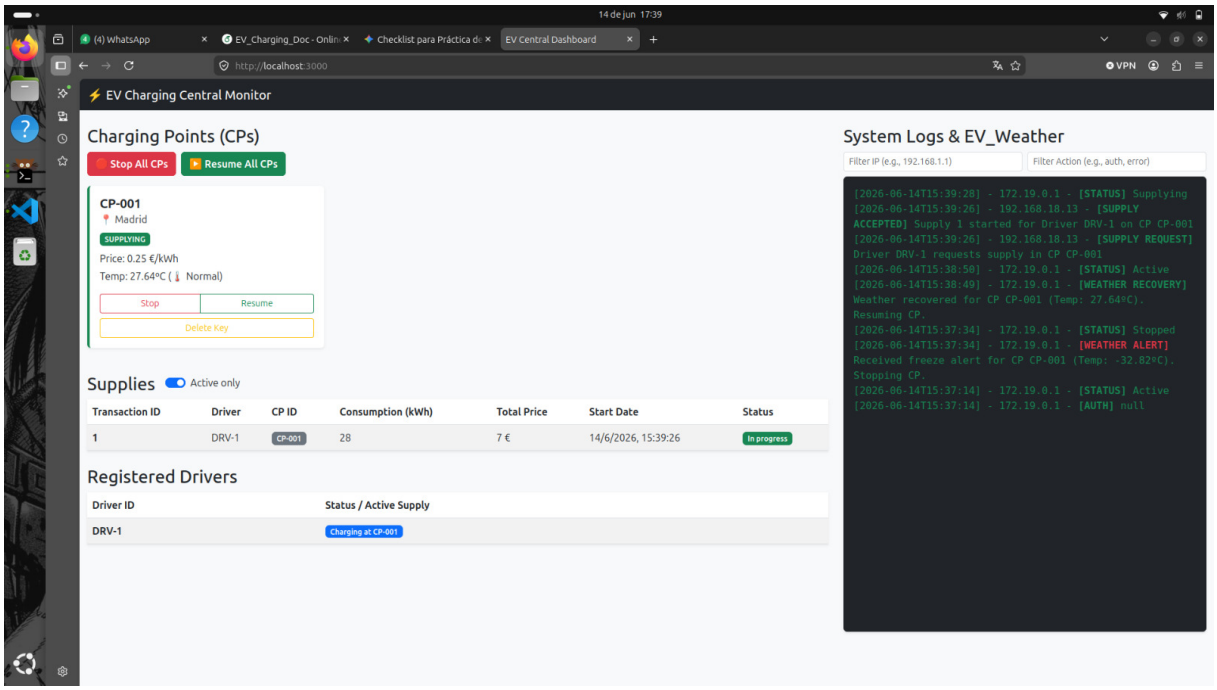

Figure 3: Front Web, supplying

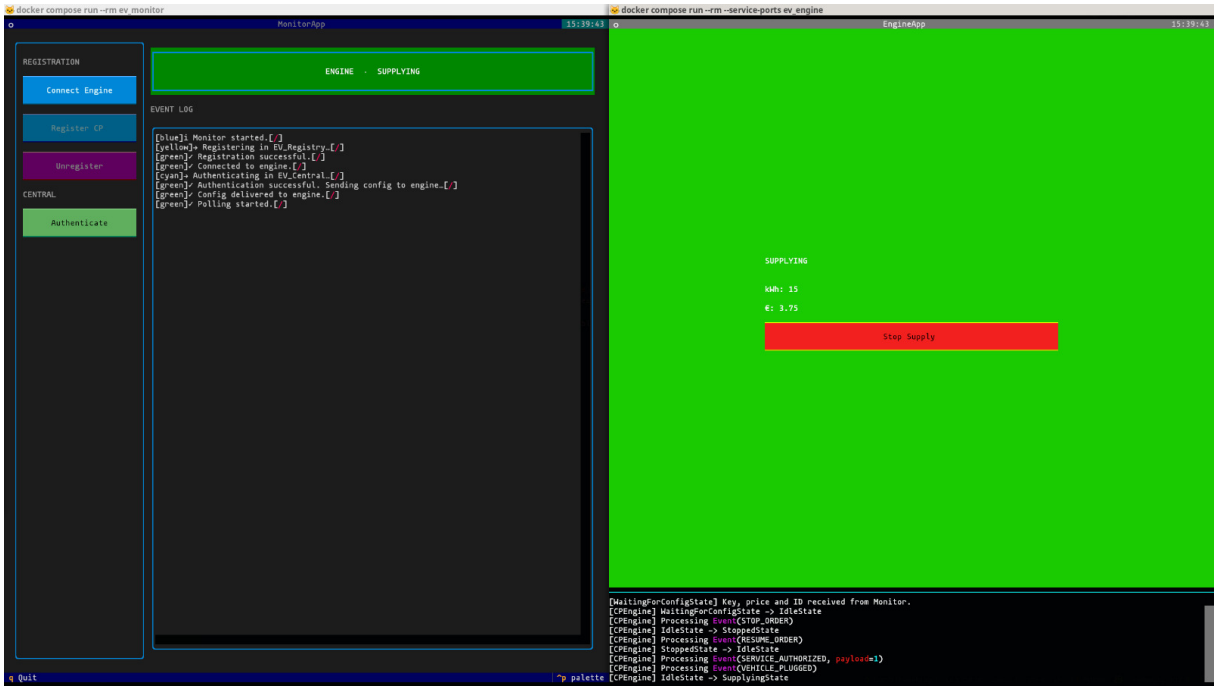

Figure 4: supplying

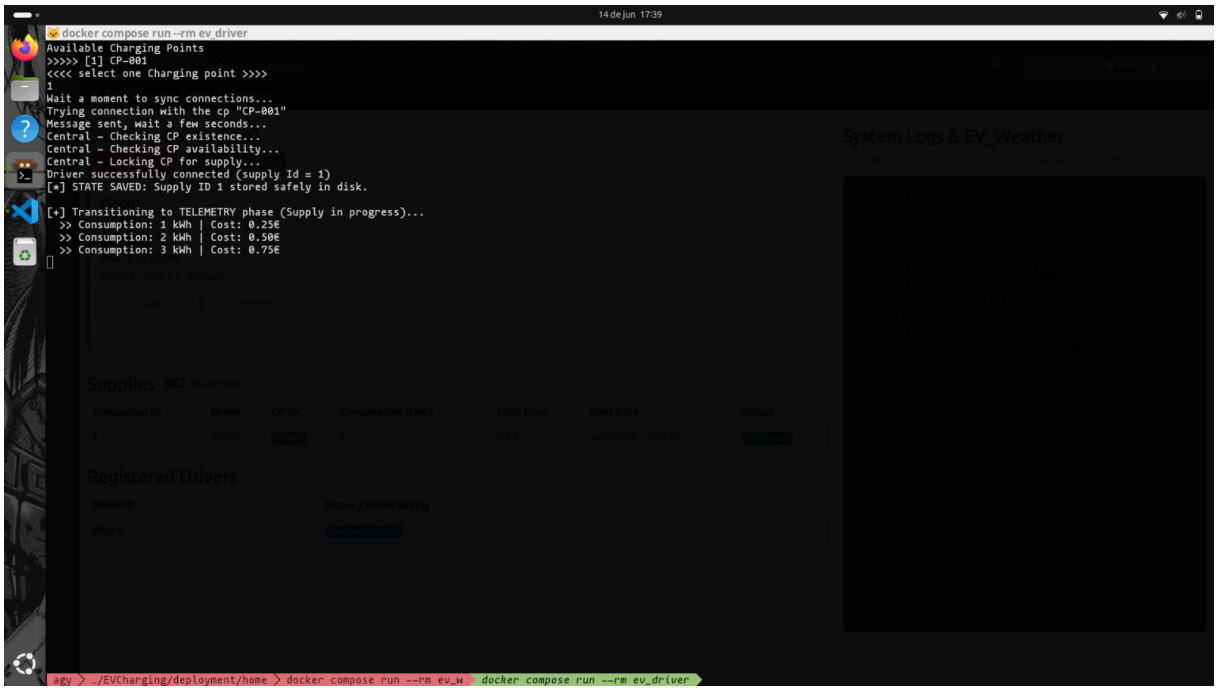

Figure 5: driver Supplying

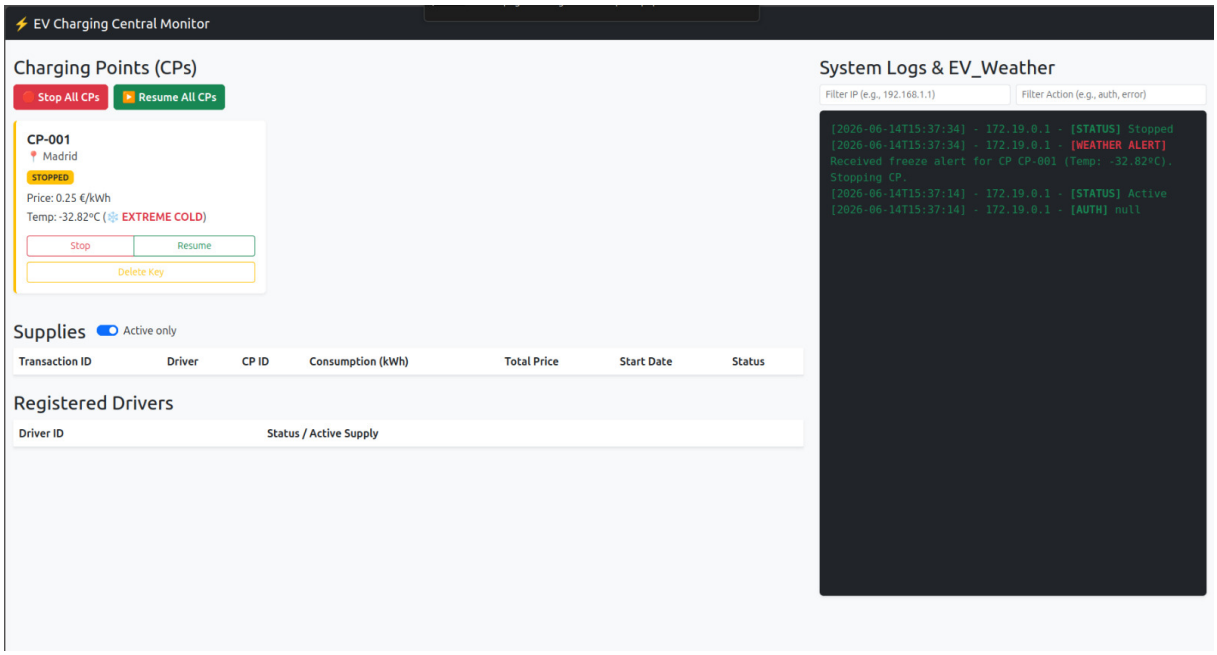

Figure 6: Front Web, cps stopped

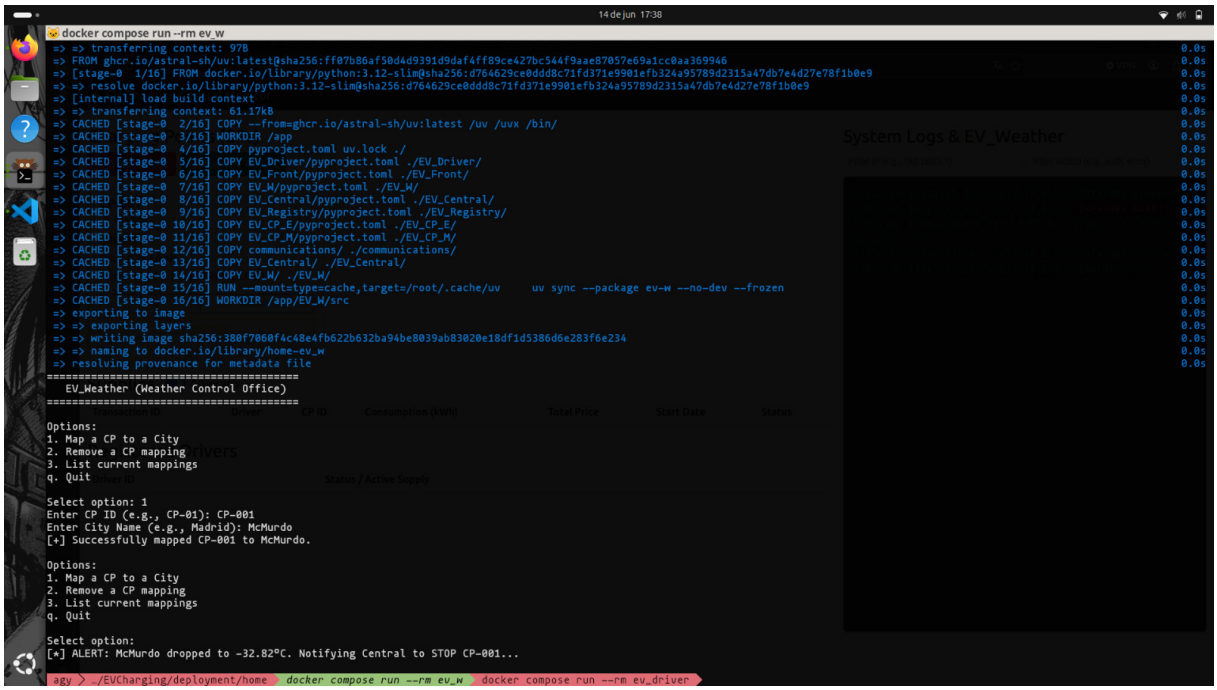

Figure 7: EV$_{W}eather$

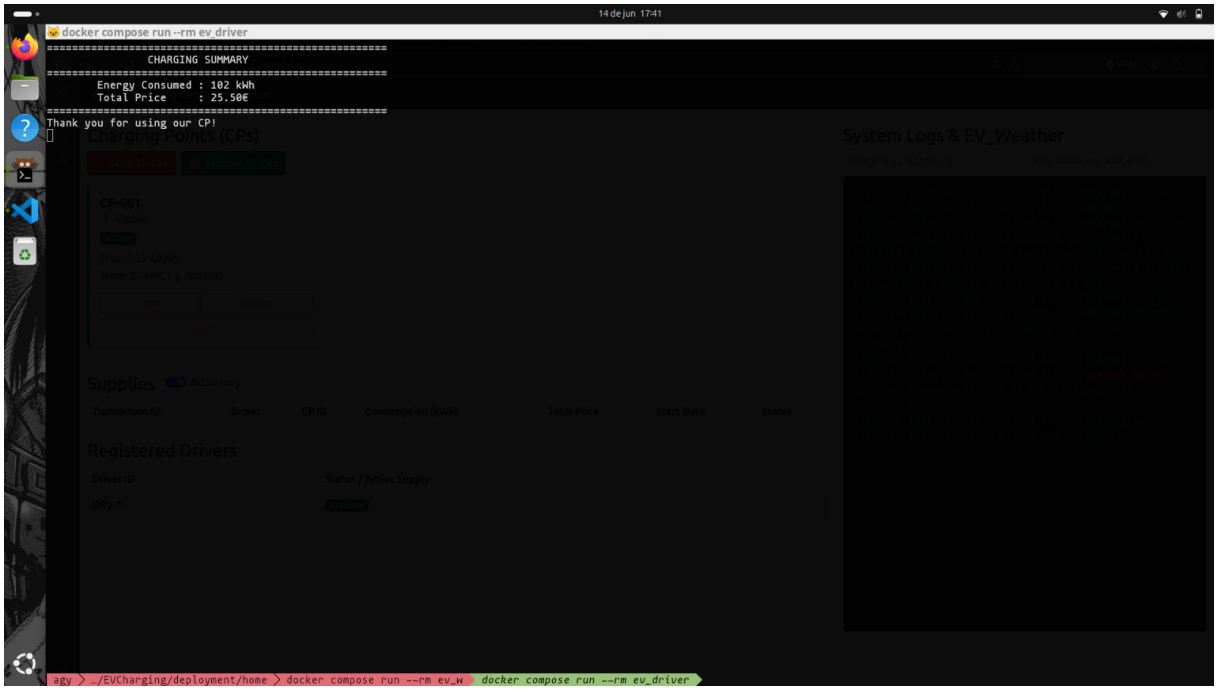

Figure 8: Driver Ticket

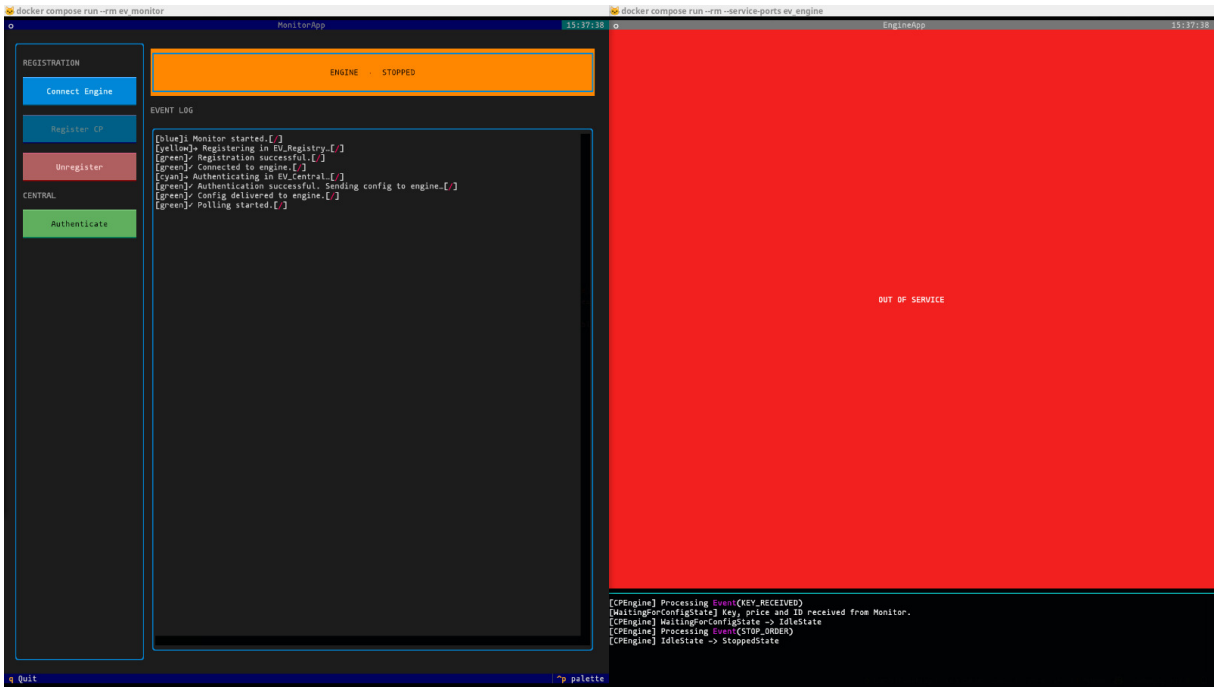

Figure 9: CP when central breakdown